valid1.js - Complete Documentation

File Location: `src/js/valid1.js`

Purpose: Real-time form validation for signup and login pages

Dependencies: None

Used by: `login.html`, `signup.html`

Overview

The `valid1.js` file provides comprehensive client-side form validation with real-time feedback. It validates user input as they type, provides inline error messages, and handles special cases like non-college email detection. The validation includes password strength checking, email format validation, and format-specific rules for Nepal phone numbers.

Regex Patterns (Module-Level Constants)

passwordRegex

```
const passwordRegex = /^(?=.*[a-z])(?=.*[A-Z])(?=.*\d)(?=.*[!@#$%^&*()_+\-=\[\]{};':"\\|,.<>\/?]).{8,}$/;
```

Pattern Requirements:

- Minimum 8 characters
- At least 1 lowercase letter: `(?=.*[a-z])`
- At least 1 uppercase letter: `(?=.*[A-Z])`
- At least 1 digit: `(?=.*\d)`
- At least 1 special character: `(?=.*[!@#$%^&*()_+\-=\[\]{};':"\\|,.<>\/?])`

Examples:

- ✅ Valid: `Password123!`, `MyPass2024@`, `Test1234#`
- ❌ Invalid: `password123!` (no uppercase), `PASSWORD123!` (no lowercase), `Password!` (no digit)

---

collegeEmailRegex

```
const collegeEmailRegex = /^[a-zA-Z]+\d+(bit|bca)\d+@kcc\.edu\.np$/i;
```

Pattern Explanation:

- `^[a-zA-Z]+` - Starts with 1+ letters
- `\d+` - Followed by 1+ digits
- `(bit|bca)` - Contains either "bit" or "bca"
- `\d+` - Followed by 1+ digits
- `@kcc\.edu\.np$` - Ends with KCC domain

Examples:

- ✅ Valid: `john1bit2@kcc.edu.np`, `sara2bca3@kcc.edu.np`
- ❌ Invalid: `john@gmail.com`, `john123@kcc.edu.np` (no bit/bca)

---

phoneRegex

```
const phoneRegex = /^(98|97)\d{8}$/;
```

Pattern Explanation:

- `^(98|97)` - Starts with 98 or 97 (Nepal phone prefixes)
- `\d{8}$` - Followed by exactly 8 digits
- Total: 10 digits

Examples:

- □ Valid: `9812345678`, `9712345678`
- □ Invalid: `9612345678` (wrong prefix), `981234567` (only 9 digits)

Helper Functions

isCollegeEmail(email)

Purpose: Check if email is a valid KCC college email.

Parameters:

- `email` (string) - Email to check

Returns: boolean

Code:

```
function isCollegeEmail(email) {
    return collegeEmailRegex.test(email);
}
```

Examples:

```
isCollegeEmail('john1bit2@kcc.edu.np');  // true
isCollegeEmail('john@gmail.com');         // false
```

---

isValidEmail(email)

Purpose: Basic email format validation (not college-specific).

Parameters:

- `email` (string) - Email to check

Returns: boolean

Code:

```
function isValidEmail(email) {
    return /^[^\s@]+@[^\s@]+\.[^\s@]+$/.test(email);
}
```

Pattern: `[text]@[domain].[extension]`

Examples:

```
isValidEmail('john@example.com');    // true
isValidEmail('john@kcc.edu.np');     // true
isValidEmail('invalid.email');       // false
```

---

isValidPassword(pw)

Purpose: Check if password meets all strength requirements.

Parameters:

- `pw` (string) - Password to check

Returns: boolean

Code:

```
function isValidPassword(pw) {
    return passwordRegex.test(pw);
}
```

Examples:

```
isValidPassword('Pass123!');  // true
isValidPassword('pass123!');  // false (no uppercase)
```

---

isValidPhone(phone)

Purpose: Validate Nepal phone number format.

Parameters:

- `phone` (string) - Phone number to check

Returns: boolean

Code:

```
function isValidPhone(phone) {
    return phoneRegex.test(phone);
}
```

Examples:

```
isValidPhone('9812345678');  // true
isValidPhone('9712345678');  // true
isValidPhone('9612345678');  // false
```

Field Validation Functions

All validation functions return error messages. Empty string means valid.

validateFullname(val)

Purpose: Validate user's full name.

Parameters:

- `val` (string) - Name to validate

Returns: string (error message, empty = valid)

Rules:

1. Required (not empty)
2. Minimum 2 characters
3. Only letters, spaces, hyphens, apostrophes

Code:

```
function validateFullname(val) {
    if (!val) return 'Full name is required.';
    if (val.length < 2) return 'Name must be at least 2 characters.';
    if (!/^[a-zA-Z\s'-]+$/.test(val)) return 'Name can only contain letters, spaces, hyphens, or apostrophes.';
    return '';
}
```

Examples:

```
validateFullname('John Doe');        // ''
validateFullname('J');               // 'Name must be at least 2 characters.'
validateFullname('John123');         // 'Name can only contain...'
validateFullname("John O'Brien");    // ''
```

---

validateEmail(val)

Purpose: Validate email with special handling for non-college emails.

Parameters:

- `val` (string) - Email to validate

Returns: object `{ error, info }`

- `error` - Blocks form submission if set
- `info` - Warning if non-college email

Rules:

1. Required
2. Valid email format
3. College email preferred (but not required)

Code:

```
function validateEmail(val) {
    if (!val) return { error: 'Email address is required.', info: '' };
    if (!isValidEmail(val)) return { error: 'Please enter a valid email address.', info: '' };
    if (!isCollegeEmail(val)) return { error: '', info: 'admin approval needed.' };
    return { error: '', info: '' };
}
```

Examples:

```
validateEmail('john@kcc.edu.np');
// { error: '', info: '' }

validateEmail('john@gmail.com');
// { error: '', info: 'admin approval needed.' }

validateEmail('invalid');
// { error: 'Please enter a valid email address.', info: '' }
```

Behavior:

- □ College email: No error, user can proceed
- ⚠ Non-college email: Shows warning but allows proceeding (with modal confirmation)
- □ Invalid format: Shows error, blocks submission

---

validateAddress(val)

Purpose: Validate residential address.

Parameters:

- `val` (string) - Address to validate

Returns: string (error message)

Rules:

1. Required
2. Minimum 5 characters

Code:

```
function validateAddress(val) {
    if (!val) return 'Address is required.';
    if (val.length < 5) return 'Please enter a more complete address.';
    return '';
}
```

Examples:

```
validateAddress('Kathmandu, Nepal');  // ''
validateAddress('NYC');               // 'Please enter a more complete address.'
validateAddress('');                  // 'Address is required.'
```

---

validatePhone(val)

Purpose: Validate Nepal phone number.

Parameters:

- `val` (string) - Phone number to validate

Returns: string (error message)

Rules:

1. Required
2. Valid Nepal phone format (98/97 + 8 digits)

Code:

```
function validatePhone(val) {
    if (!val) return 'Phone number is required.';
    if (!isValidPhone(val)) return '(98XXXXXXXX or 97XXXXXXXX).';
    return '';
}
```

Examples:

```
validatePhone('9812345678');  // ''
validatePhone('123456');      // '(98XXXXXXXX or 97XXXXXXXX).'
validatePhone('');            // 'Phone number is required.'
```

---

validatePassword(val)

Purpose: Validate password strength.

Parameters:

- `val` (string) - Password to validate

Returns: string (error message for first failed requirement)

Rules:

1. Minimum 8 characters
2. At least 1 uppercase letter (A-Z)
3. At least 1 lowercase letter (a-z)
4. At least 1 digit (0-9)
5. At least 1 special character (!@#$%...)

Code:

```
function validatePassword(val) {
    if (!val) return 'Password is required.';
    if (val.length < 8) return 'Password must be at least 8 characters.';
    if (!/[A-Z]/.test(val)) return 'Add at least one uppercase letter.';
    if (!/[a-z]/.test(val)) return 'Add at least one lowercase letter.';
    if (!/\d/.test(val)) return 'Add at least one digit (0-9).';
    if (!/[!@#$%^&*()_+\-=\[\]{};':"\\|,.<>\/?]/.test(val)) return 'Add at least one symbol (!@#$%…).';
    return '';
}
```

Examples:

```
validatePassword('Pass123!');      // ''
validatePassword('Pass');          // 'Password must be at least 8 characters.'
validatePassword('password123!');  // 'Add at least one uppercase letter.'
validatePassword('PASSWORD123!');  // 'Add at least one lowercase letter.'
validatePassword('Password!');     // 'Add at least one digit (0-9).'
validatePassword('Password1');     // 'Add at least one symbol...'
```

---

validateConfirmPassword(val, pw)

Purpose: Validate password confirmation matches original password.

Parameters:

- `val` (string) - Confirm password field value
- `pw` (string) - Original password field value

Returns: string (error message)

Rules:

1. Required (not empty)
2. Must match password field exactly

Code:

```
function validateConfirmPassword(val, pw) {
    if (!val) return 'Please confirm your password.';
    if (val !== pw) return 'Passwords do not match.';
    return '';
}
```

Examples:

```
validateConfirmPassword('Pass123!', 'Pass123!');  // ''
validateConfirmPassword('Pass123!', 'Pass123');   // 'Passwords do not match.'
validateConfirmPassword('', 'Pass123!');          // 'Please confirm your password.'
```

UI Feedback Functions

setFeedback(fieldName, msg, level = 'error')

Purpose: Show or clear validation feedback under form field.

Parameters:

- `fieldName` (string) - Field identifier (matches `data-feedback` attribute)
- `msg` (string|null) - Message to display (null/empty = clear)
- `level` (string) - 'error' or 'info' (affects styling)

Returns: void

What it does:

1. Finds element with `[data-feedback=fieldName]`
2. Sets text content to message
3. Applies CSS class based on level (error = red, info = yellow)
4. Empty message clears the feedback

Code:

```
function setFeedback(fieldName, msg, level = 'error') {
    const el = document.querySelector(`[data-feedback="${fieldName}"]`);
    if (!el) return;
    if (msg) {
        el.textContent = msg;
        el.className = `field-feedback ${level}`;
    } else {
        el.textContent = '';
        el.className = 'field-feedback';
    }
}
```

HTML Example:

```
<input id="email" type="email">
<p data-feedback="email"></p>

<!-- After setFeedback('email', 'Invalid format', 'error'): -->
<!-- <p data-feedback="email" class="field-feedback error">Invalid format</p> -->
```

Usage:

```
// Show error
setFeedback('email', 'Invalid email format', 'error');

// Show warning
setFeedback('email', 'Needs admin approval', 'info');

// Clear message
setFeedback('email', '');
```

---

setInputState(input, hasError)

Purpose: Add visual indicator (border color) to input field.

Parameters:

- `input` (HTMLElement) - Input element
- `hasError` (boolean) - true for error state

Returns: void

Code:

```
function setInputState(input, hasError) {
    if (!input) return;
    input.classList.toggle('input-error',   hasError);
    input.classList.toggle('input-success', !hasError);
}
```

CSS:

```
.input-error {
    border-color: #e74c3c;  /* Red */
    background-color: #ffe6e6;
}

.input-success {
    border-color: #27ae60;  /* Green */
    background-color: #e8f8f0;
}
```

Example:

```
const emailInput = document.getElementById('email');

if (isValidEmail(emailInput.value)) {
    setInputState(emailInput, false);  // Green border
} else {
    setInputState(emailInput, true);   // Red border
}
```

Form State Management

Module Variable: nonCollegeAcknowledged

```
let nonCollegeAcknowledged = false;
```

Purpose: Track if user has confirmed signup with non-college email.

Set to true when: User clicks "Wait for Approval" in non-college email modal.

---

updateSubmitButton()

Purpose: Enable/disable signup submit button based on form validity.

Parameters: None
Returns: void

What it does:

1. Gets all form field values
2. Runs each validator
3. Disables button if ANY field has error
4. Enables button only when ALL fields valid

Code Logic:

```
function updateSubmitButton() {
    const btn = document.getElementById('signupSubmitBtn');
    if (!btn) return;

    const fullname = document.getElementById('fullname')?.value.trim() ?? '';
    const email    = document.getElementById('email')?.value.trim() ?? '';
    const address  = document.getElementById('address')?.value.trim() ?? '';
    const phone    = document.getElementById('phone')?.value.trim() ?? '';
    const password = document.getElementById('password')?.value ?? '';
    const confirm  = document.getElementById('confirm-password')?.value ?? '';

    const allValid =
        !validateFullname(fullname) &&
        !validateEmail(email).error &&
        isValidEmail(email) &&
        !validateAddress(address) &&
        !validatePhone(phone) &&
        !validatePassword(password) &&
        !validateConfirmPassword(confirm, password);

    btn.disabled = !allValid;
}
```

Execution Flow:

```
User types in any field
    ↓
updateSubmitButton() triggered
    ↓
Get all field values
    ↓
Validate each field
    ↓
Check if all valid
    ↓
Enable/disable submit button
```

Non-College Email Modal

showNonCollegeModal(email, onProceed)

Purpose: Show confirmation modal for non-college email signup.

Parameters:

- `email` (string) - The non-college email entered
- `onProceed` (function) - Callback if user proceeds

Returns: void

What it does:

1. Creates overlay modal with warning
2. Shows email entered
3. Explains admin approval required
4. Two buttons: "Wait for Approval" / "Use Different Email"
5. If approve: sets `nonCollegeAcknowledged = true`, calls onProceed
6. If cancel: removes modal, focuses email input

Code:

```
function showNonCollegeModal(email, onProceed) {
    document.getElementById('nonCollegeModal')?.remove();

    const modal = document.createElement('div');
    modal.id = 'nonCollegeModal';
    modal.className = 'nc-modal-overlay';
    modal.innerHTML = `
        <div class="nc-modal">
            <h3>Non-College Email Detected</h3>
            <p>The email <strong>${email}</strong> is not a registered KCC college email.</p>
            <p>You can still sign up, but your account will be <strong>pending admin approval</strong> before you can log in.</p>
            <div class="nc-modal-actions">
                <button id="ncProceed" class="btn btn-primary">Wait for Approval</button>
                <button id="ncCancel"  class="btn btn-secondary">Use a Different Email</button>
            </div>
        </div>
    `;
    document.body.appendChild(modal);

    document.getElementById('ncProceed').addEventListener('click', () => {
        nonCollegeAcknowledged = true;
        modal.remove();
        onProceed();
    });

    document.getElementById('ncCancel').addEventListener('click', () => {
        modal.remove();
        document.getElementById('email').focus();
    });
}
```

User Experience:

```
User enters non-college email
    ↓
Shows warning modal
    ↓
User clicks "Wait for Approval"
    ↓
Sets flag, submits form
    ↓
Account created, pending approval
```

Form Submission Handlers

handleLoginSubmit(event)

Purpose: Validate login form before submission.

Parameters:

- `event` (Event) - Form submit event

Returns: boolean (false to prevent submit)

What it does:

1. Checks email and password not empty
2. Shows alerts for missing fields
3. Returns false to prevent submit if invalid

Code:

```
function handleLoginSubmit(event) {
    const email    = document.getElementById('email')?.value.trim();
    const password = document.getElementById('password')?.value;

    if (!email || !password) {
        event.preventDefault();
        if (!email)    alert('Please enter your email.');
        else if (!password) alert('Please enter your password.');
        return false;
    }
    return true;
}
```

HTML Usage:

```
<form onsubmit="return handleLoginSubmit(event)">
    <input id="email" type="email" required>
    <input id="password" type="password" required>
    <button type="submit">Login</button>
</form>
```

---

handleSignupSubmit(event)

Purpose: Comprehensive validation before signup submission.

Parameters:

- `event` (Event) - Form submit event

Returns: boolean (false to prevent submit)

What it does:

1. Gets all form values
2. Runs all validators as safety check
3. Updates feedback elements
4. Checks for errors
5. If non-college email AND not acknowledged:
    - Prevents form submission
    - Shows non-college modal
    - Waits for user confirmation
6. Otherwise allows submission

Code Structure:

```
function handleSignupSubmit(event) {
    // Validate all fields
    const fullname = document.getElementById('fullname').value.trim();
    const email = document.getElementById('email').value.trim();
    // ... get all fields

    // Update feedback for each field
    setFeedback('fullname', validateFullname(fullname));
    // ... set all feedbacks

    // Check for errors
    const hasErrors = /* check all validators */;

    if (hasErrors) {
        event.preventDefault();
        return false;
    }

    // Handle non-college email
    if (!isCollegeEmail(email) && !nonCollegeAcknowledged) {
        event.preventDefault();
        showNonCollegeModal(email, () => {
            // Re-submit form
            event.target.submit();
        });
        return false;
    }

    return true;
}
```

Complete Signup Flow:

```
User clicks signup
    ↓
Fills form with values
    ↓
Clicks submit button
    ↓
handleSignupSubmit() triggered
    ↓
Validates all fields
    ↓
Shows error messages if any
    ↓
If college email: proceed
If non-college: show modal
    ↓
User confirms
    ↓
Form submits to signup.php
    ↓
Backend creates account
    ↓
User logged in
```

Page Initialization

Setting up Real-Time Validation

Typically initialized on page load:

```
document.addEventListener('DOMContentLoaded', () => {
    // Setup real-time validation
    const validationFields = ['fullname', 'email', 'password', 'confirm-password', 'phone', 'address'];

    validationFields.forEach(field => {
        document.getElementById(field)?.addEventListener('input', updateSubmitButton);
    });

    // Email field also updates feedback
    document.getElementById('email')?.addEventListener('input', function() {
        const emailResult = validateEmail(this.value);
        setFeedback('email', emailResult.error || emailResult.info,
                   emailResult.error ? 'error' : 'info');
    });
});
```

Integration Points

Works with modal1.js

Modal content includes signup forms with validation enabled.

Works with both_homepage.js

Email validation happens when user creates account before accessing events.

Server-Side Validation

Frontend validation is convenience; backend in `signup.php` validates again:

```
// Backend also validates
if (!isValidEmail($email)) {
    die('Invalid email format');
}
```

Summary

valid1.js provides:

Feature	Purpose
Real-time Validation	Instant feedback as user types
College Email Handling	Allows non-college with approval
Password Strength	Enforces strong passwords
Nepal Phone Format	Validates local phone numbers
UI Feedback	Shows errors and warnings inline
Form State	Enables/disables submit button

This creates a user-friendly signup/login experience with clear guidance on what's needed.

Last Updated: May 22, 2026
File Size: ~200 lines of code
Complexity: Medium-High
Criticality: High (security-adjacent)